正则表达式，又称规则表达式。（英语：Regular Expression，在代码中常简写为regex、regexp或RE），计算机科学的一个概念。正则表达式通常被用来检索、替换那些符合某个模式(规则)的文本。

正则表达式匹配：

第一种匹配方式：

var reg = /pattern/modifier
/模式（规定怎么去匹配的）/ 修饰符

第二种匹配方式：

var reg = new RegExp("pattern" , "modifier");

修饰符：g为全局内容（global）　i忽略大小写

修饰符	含义	描述
i	ignore - 不区分大小写	将匹配设置为不区分大小写，搜索时不区分大小写: A 和 a 没有区别。
g	global - 全局匹配	查找所有的匹配项。
m	multi line - 多行匹配	使边界字符 `^` 和 `$` 匹配每一行的开头和结尾，记住是多行，而不是整个字符串的开头和结尾。
s	特殊字符圆点 `.` 中包含换行符 `\n`	默认情况下的圆点 `.` 是 匹配除换行符 `\n` 之外的任何字符，加上 `s` 修饰符之后，`.` 中包含换行符 \n。

```
var str = 'hello';
var reg = /l/;
 console.log( str.match( reg ) );
 //匹配不到则返回空null
```

RegExp 对象的方法

test()

test() 方法检索字符串中的指定值。

返回值是 true 或 false。

```
var str = 'helloe';
var reg = /e/;
console.log( reg.test(str) );//true
```

exec()

//RegExp.exec(string)
//匹配字符串返回首次匹配结果
var str = 'helloe';
var reg = /e/g;
console.log(reg.exec(str)); //返回对应的内容

元字符

\w :匹配字母，数字，_下划线

\W :匹配非字母，非数字，非_下划线

var reg = /\w/
var reg = /\W/

```
/*
    \w   匹配单词 数字 _
    \W   匹配非单词 非数字 _
*/
var str = 'A';
var reg = /\w/;
console.log( reg.test(str) );
```

\d : 匹配数字;

\D : 匹配非数字;

var reg = /\d/
var reg = /\D/

```
var str = 'A';
var reg = /\D/;
console.log( reg.test(str) );
```

\s :匹配空格;

\S :匹配非空格;

var reg = /\s/
var reg = /\S/

```
var str="Is this all there is?";
var reg=/\s/;
console.log( str.match( reg ) )
```

\b单词边界与\B 非单词边界(间隙)

```
var str = 'hello';
var reg = /\b/;
console.log( str.match(reg) );
```

特殊转移字符

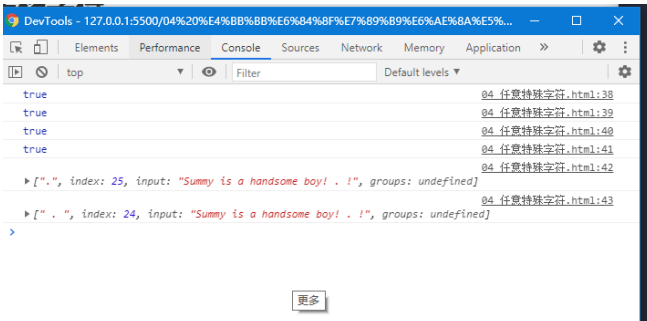
- （点） 任意字符

任意字符，除了换行符\n 制表符\r匹配不到，其它的内容都可以匹配到；
\ 反斜框有转义的功能；

匹配反斜杠

```
var str = '\\';
var reg = /./g;
console.log( str )
console.log( reg.test( str ) );

var str = '\\';
var reg = /\\/g;
console.log( str )
console.log( reg.test( str ) );
```




```
<script>

    /*
        .代表的是任意字符;
    */
    var str = 'Summy is a handsome boy! . !';
    var str1 = '///'
    var str2 = '<script><\/script>'
    var reg = /./;
    var reg1 = /!/;
    var reg2 = /\//;   // \ 代表的是转义;
    var reg3 = /\./;
    var reg4 = / \. /;


    console.log( reg.test( str ) );
    console.log( reg.test( str1 ) );
    console.log( reg2.test( str1 ) );
    console.log( reg3.test( str ) );
    console.log( str.match( reg3) )
    console.log( str.match( reg4) )
```

量词

n{num}确定个数

n{num1,}大于或等于num1 ，必须是需要连着的；
var reg2 = /B{3,}/; //[3,- ∞) 大于或等于2

n{num1,num2}处于num1与num2之间[包含num1,num2] ，必须是需要连着的；

至少出现num1次，最多出现num2次

^n 以n开头

^表示起始位置；/^a/.test(‘str’);表示是否以a开头

```
var str = 'a123456789';
var reg = /^\d+/;//非数字开头    //以字母开头，一个或多个数字;

console.log( reg.test( str ) );

var reg = /^[a-z]\d+/;  //匹配以数字开头
console.log( reg.test( str ) );
```

n$ 以n结尾

```
var str = '123456d';
var reg = /^\d+d$/;
console.log( reg.test( str ) );
```

字集

var reg3 = /[a-zA-Z]/ // abcdefg....z
var reg4 = /[A-z0-9]/ // abcdefg....z

[\u4e00-\u9fa5] //用来判断是不是中文

区间

特殊情况，0会匹配到边界

```
var str = 'd8888d';
var reg = /\d{0,1}/
console.log(reg.test(str)); //true
console.log(str.replace(reg, '6')) //6d8888d
```

\n 换行符

贪婪模式与惰性模式

贪婪模式与惰性模式难点解析：

贪婪模式：匹配尽可能多的字符

首先比对整个字符串是否匹配，如果匹配不成功则去掉字符串最后一个字符再继续进行匹配，以此类推直到匹配成功或不剩字符才停止。

惰性模式：匹配尽可能少的字符

首先从字符串第一个字符开始匹配，如果匹配成功则停止匹配，否则加入第二个字符继续匹配，依此类推直到匹配成功或不剩字符才停止。

```
贪婪模式【默认的】：
    n+
    n*
    n{0,}
    n{0,3}
    n{3}
惰性模式:
    n+?
    n*?
    n{0,}?
    n{0,3}?
    n{3}?
```

```
var str = '0123456789';
var reg = /\d+/;  //至少匹配一个,
var reg = /\d*/;  //至少匹配零个,
var reg = /\d{0,}/;  //0123456789
var reg = /\d{3}/;  //012

var reg = /\d+?/;  // 0
var reg = /\d*?/;  // ""
var reg = /\d?/;  //0
var reg = /\d{2,}?/;

console.log( str.match( reg ) )
```

分组

选择性 ： | 或者 （）分组

var str = "我喜欢高富帅";
var reg = /我喜欢(高|富|帅)
console.log(reg.test('我喜欢胖高'))//false
console.log(reg.test('我喜欢富胖'))//true

从左往右匹配，满足分组中的一项则为true

var str = "2021-04-06"
var reg = /(\d{4})-(\d{2})-(\d{2})/
console.log(str.match(reg))//匹配到这条串后可拿到里面分组值
console.log(RegExp.$1) //拿到第一个分组
console.log(RegExp.$2) //拿到第二个分组
console.log(RegExp.$3) //拿到第三个分组
//var reg = /(summy) is \1/
// \1 代表的是第一个分组

换位置 替换:

str = str.replace(reg , "$1/$2/$3")// 2021/04/06
str = str.replace(reg , "$2/$3-$1")//04/06-2021

```
var str = '岁月是个大傻逼';
var reg = /大(帅逼|傻逼)/g
// console.log( reg.test( str ) )
// console.log( str.match( reg ))
str = str.replace( reg , function( word ){
    console.log( word )
    return word.replace( /./g , "*" )

} )
console.log( str )//岁月是个***
```

如果有多个分组，则会先打开最外层括号，再依次从左到右打开括号。

```
var str = 'suiyue is a shuaibi'
var reg = /((suiyue) is (a (shuaibi)))/

// console.log( str.match( reg ) )
console.log( str.match( reg ) )
```

```
02 分组.html:71
▸ (5) ["suiyue is a shuaibi", "suiyue is a shuaibi", "suiyue", "a shuaibi", "shuaibi", index: 0, input: "suiyue is a shuaibi", groups: undefined]
```

捕获型 ： ()

RegExp.$1

非捕获型 ： (?:)

RegExp.$1拿不到第一个满足的；

正向前瞻(断言)型:

(?=shuaibi) 能匹配到则获取它之前的值

反向前瞻(断言)型:
(?!shuaibi) 取反:能匹配到则不获取它之前的值

正向
/suiyue is a (?=shuaibi)/
反向
/suiyue is a (?!shuaibi)/

```
//邮箱:
/*
    1605716666@qq.com
*/
var reg = /^(\w{5,12})@(qq|sina|foxmail).(com|cn)/
```

https://c.runoob.com/front-end/854 正则表达式在线测试